



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

www.rvce.edu.in
Tel: +91-80-68188100
+91-80-68188111
+91-80-68188112

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SecureCam: Zero-Trust Surveillance Access System using PAKE Authentication Network Programming and Security – CS362IA

Experiential Learning (Lab) ®

REPORT

Submitted by

Nikhil Parashuram Bakale 1RV23CS154

Prakhar Parashar 1RV23CS173

Piyush Kheria 1RV23CS166

Under the guidance of

Prof. Mekhala Vinod Purohit

Assistant Professor

Department of CSE

In partial fulfilment for the award of degree of

Bachelor of Engineering

in

Department of Computer Science and Engineering

2025-2026





RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

CERTIFICATE

Certified that the project work titled *SecureCam: Zero-Trust Surveillance Access System using PAKE Authentication* is carried out by **Nikhil Parashuram Bakale (1RV23CS154), Prakhar Parashar (1RV23CS173), Piyush Kheria (1RV23CS166)** who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfilment for the award of degree of **Bachelor of Engineering in Computer Science and Engineering (CSE)** of the **Visvesvaraya Technological University**, Belagavi during the academic year 2025-2026. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the report deposited in the departmental library. The report has been approved as it satisfies the academic requirements in respect of experiential learning work prescribed by the institution for the said degree.

Signature of Guide
Guide Name:

Signature of Head of the Department /
Program Coordinator

External Viva

Name of Examiners

Signature with Date

1

2





RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

DECLARATION

We, **Nikhil Parashuram Bakale, Prakhar Parashar, Piyush Kheria**, students of Sixth semester B.E., department of Computer Science and Engineering, RV College of Engineering, Bengaluru, hereby declare that Experiential Learning (Lab) titled ***SecureCam: Zero-Trust Surveillance Access System using PAKE Authentication*** has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering in Computer Science and Engineering (CSE)** during the academic year 2025-26

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering®, Bengaluru and we will be one of the authors of the same.

Place: Bengaluru

Date:

Name

Signature

Nikhil Parashuram Bakale (1RV23CS154)

Prakhar Parashar (1RV23CS173)

Piyush Kheria (1RV23CS166)



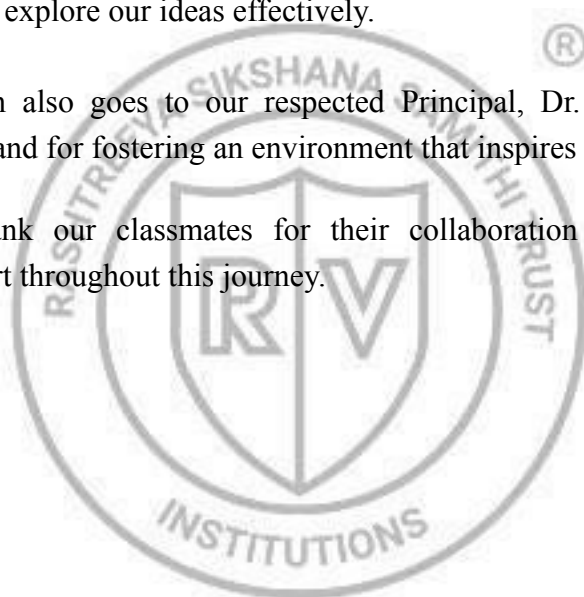
ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our esteemed guide, Prof Mekhala Vinod Purohit , Assistant Professor, Department of CSE, RVCE for her invaluable guidance, support, and encouragement throughout this project. Her insights and mentorship were instrumental in bringing this phase to a successful completion.

We also extend our heartfelt thanks to Dr. Shantha Rangaswamy, Head of the Department, CSE, RVCE and Dr. Ramakanth Kumar P, Professor and Dean, CSE Cluster, RVCE for providing us with the necessary facilities and resources for conducting our implementation and testing. Their support made it possible for us to explore our ideas effectively.

Our sincere appreciation also goes to our respected Principal, Dr. K N Subramanya, for his constant encouragement and for fostering an environment that inspires research and innovation.

Lastly, we wish to thank our classmates for their collaboration and our families for their unwavering moral support throughout this journey.





ABSTRACT

Surveillance systems are widely used in residential complexes, campuses, offices, and restricted facilities for continuous monitoring and incident response. However, most traditional surveillance portals depend on a simple login-based model where authentication is often treated as authorization. Once a user logs in, they may receive broad or long-term access to sensitive camera feeds. If a session token is stolen or misused, camera access may continue without proper re-verification, and administrators may find it difficult to track who viewed which feed and when.

SecureCam: Zero-Trust Surveillance Access System using PAKE Authentication addresses this problem by implementing a secure, role-based, and audit-ready camera access platform. The system uses Password-Authenticated Key Exchange (PAKE), specifically SPAKE2, to authenticate users without directly transmitting the password during login. After successful authentication, the backend issues a JSON Web Token (JWT), but this token proves only the user's identity and role. Camera viewing is not granted automatically; residents, viewers, or security guards must request access to specific cameras, and an administrator must approve the request for a limited duration.

The core contribution of the system is its zero-trust access workflow. After approval, the backend creates a time-bound camera assignment. When the user opens an assigned camera, the frontend requests a camera-scoped capability token bound to the user, camera, assignment, permission, and expiry time. Before the feed is unlocked, this token must be validated using a fresh one-time nonce. If the same nonce is reused, the backend rejects the request and records a `REPLAY_ATTACK_DETECTED` security event, preventing replay attacks and reuse of captured authorization requests.

The system is implemented using a React and TypeScript frontend, a FastAPI backend, SQLite database, JWT-based identity sessions, SPAKE2-based PAKE authentication, capability tokens, and nonce replay protection. It provides role-specific dashboards for administrators, residents, viewers, and security guards, along with audit logs and a security dashboard. The current camera preview uses optimized frame upload and polling with MJPEG support for demonstration. While WebRTC-based end-to-end streaming and real AI-based detection are future enhancements, the implemented system demonstrates practical Network Programming and Security concepts such as secure authentication, least-privilege authorization, replay prevention, RBAC, and accountability in a realistic surveillance use case.



TABLE OF CONTENTS

Abstract

Chapter 1: Introduction	1–2
1.1 Overview of the Problem Statement	
1.2 Significance of the Problem	
1.3 Objectives of the Proposed Solution	
1.4 Scope of the Project	
Chapter 2: Solution Design	3–6
2.1 Overall System Design and Architecture	
2.2 Authentication and Authorization Design	
2.3 Role-Based Access Control Design	
2.4 Database Design and Data Structures Used	
2.5 Flow Chart of the Proposed System	
2.6 Description of the Flow Chart	
Chapter 3: Implementation Details	7–19
3.1 Implementation Approach	
3.2 Modular Design and Code Organization	
3.3 PAKE Authentication Implementation	
3.4 Capability Token and Nonce Replay Protection	
3.5 Camera Access and Streaming Implementation	
3.6 Audit Logs and Security Dashboard Implementation	
3.7 Coding Best Practices and Maintainability	
Chapter 4: Results and Discussion	10–17
4.1 Execution Results and Validation	
4.2 Security Test Cases and Edge Cases	
4.3 Replay Attack Detection and Access Revocation Results	
4.4 Comparative Analysis with Traditional Surveillance Systems	
4.5 Discussion and Scope for Improvements	
Chapter 5: Conclusion and Future Scope	18–19
5.1 Summary of Findings	
5.2 Real-World Applicability	
5.3 Future Enhancements and Innovations	

References

20

Appendix

- A. Screenshots of Project Execution
- B. Backend Test Results
- C. API Endpoint Summary
- D. Plagiarism Report



CHAPTER 1

INTRODUCTION

1.1 Overview of the Problem Statement

Surveillance systems are widely used in residential apartments, college campuses, offices, industries, and restricted facilities for continuous monitoring and incident response. These systems handle sensitive camera feeds that may include private spaces, entry points, people's movement, and security-critical areas. Therefore, camera access must be controlled carefully. However, many traditional surveillance portals still follow a simple login-based model where a user enters credentials, receives a session token, and is then allowed to access camera feeds mainly based on that login session.

The major limitation of this approach is that authentication is often treated as complete authorization. A login token such as a JWT can prove the user's identity, but it does not necessarily prove that the user should be allowed to view a particular camera at a particular time. If such a token is stolen, misused, or remains active for too long, unauthorized users may gain access to sensitive surveillance feeds without fresh approval or additional verification.

Another issue is the lack of temporary and camera-specific access control. In real scenarios, a resident or security guard may need camera access only for a limited purpose, such as checking an entry gate or verifying an incident. Providing broad or permanent access increases privacy risks and makes auditing difficult. Administrators must be able to track who requested access, who approved it, which camera was viewed, and when the access expired or was revoked.

This project addresses these challenges by designing **SecureCam**, a zero-trust surveillance access system using PAKE authentication. The system separates authentication from camera authorization. Users authenticate securely using PAKE, while the JWT issued after login proves only identity and role. Actual camera viewing requires an admin-approved assignment, a camera-scoped capability token, and fresh nonce validation to prevent replay attacks. This creates a secure, time-bound, auditable, and least-privilege surveillance access workflow.

1.2 Significance of the Problem

The security of surveillance systems is directly connected to privacy, safety, and accountability. Camera feeds often capture sensitive areas such as building entrances, corridors, parking spaces, offices, and residential surroundings. If access to these feeds is given too broadly after a simple login, unauthorized viewing can occur without immediate detection. In environments like apartments, colleges, and offices, this can lead to privacy violations and misuse of surveillance data.

Beyond privacy, the security of authentication and authorization is equally important. A stolen password or misused JWT can allow an attacker or unauthorized user to access camera feeds if the system treats login as complete permission. Without camera-specific authorization, replay protection, and access expiry, even a valid user may retain access longer than required or reuse old authorization data to view a feed again.

A secure surveillance framework must therefore separate identity verification from actual camera access. By using PAKE authentication, admin-approved assignments, capability tokens, nonce replay protection, and audit logs, SecureCam ensures that camera access is temporary, specific, and traceable. This makes the system more suitable for real-world surveillance scenarios where privacy, least-privilege access, and accountability are essential.

1.3 Objectives of the Proposed Solution

- The primary objective of **SecureCam** is to demonstrate a complete, working implementation of a zero-trust surveillance access system using PAKE authentication. Specifically, the project aims to:
- Implement a secure PAKE-based authentication mechanism using SPAKE2 so that user passwords are not directly transmitted during the login process.
- Issue JWT tokens only for user identity and role verification, while preventing JWTs from directly granting camera viewing access.
- Implement role-based access control for admin, resident, security guard, and viewer roles with proper separation of permissions.
- Develop a request and approval workflow where residents or guards can request camera access and administrators can approve or reject access for a limited duration.
- Implement time-bound camera assignments and camera-scoped capability tokens to ensure that users can view only authorized cameras.
- Add nonce-based replay attack protection so that reused authorization requests are rejected and logged as security events.
- Provide audit logs and a security dashboard to record login events, access approvals, revocations, camera views, replay attacks, and unauthorized access attempts.

By fulfilling these objectives, SecureCam establishes that secure authentication, least-privilege authorization, replay protection, and auditability can be combined into a lightweight surveillance access system suitable for academic demonstration and real-world security-focused use cases.

1.4 Scope of the Project

The scope of **SecureCam** is to develop a secure surveillance access control system where camera feeds are not unlocked merely after login. The project implements PAKE-based authentication, JWT-based identity verification, role-based access control, admin-approved temporary assignments, camera-scoped capability tokens, nonce replay protection, audit logs, and a security dashboard. It supports admin, resident, security guard, and viewer roles, along with demo camera access through local webcam frame upload and MJPEG-based streaming.

The project is focused on demonstrating core Network Programming and Security concepts such as secure authentication, least-privilege authorization, replay attack prevention, time-bound access, and auditability. Production-grade WebRTC streaming, real AI-based detection, cloud deployment, mobile application support, and enterprise-scale multi-tenant architecture are considered future enhancements.

CHAPTER 2

SOLUTION DESIGN

2.1 Overall System Design and Architecture

SecureCam is designed as a web-based zero-trust surveillance access system with separate components for user interaction, authentication, authorization, camera access, and audit monitoring. The React frontend provides the login page and role-based dashboards, while the FastAPI backend acts as the trusted security layer for PAKE login, JWT identity verification, RBAC, assignments, capability tokens, nonce replay protection, and audit logging.

The system stores users, cameras, access requests, assignments, used nonces, audit logs, and security events in an SQLite database. Camera feeds are handled through local webcam frame upload or MJPEG proxying and are displayed only after the complete zero-trust flow is validated. Thus, camera access is not granted merely after login, but only after identity verification, admin approval, active assignment checking, capability token validation, and fresh nonce verification.

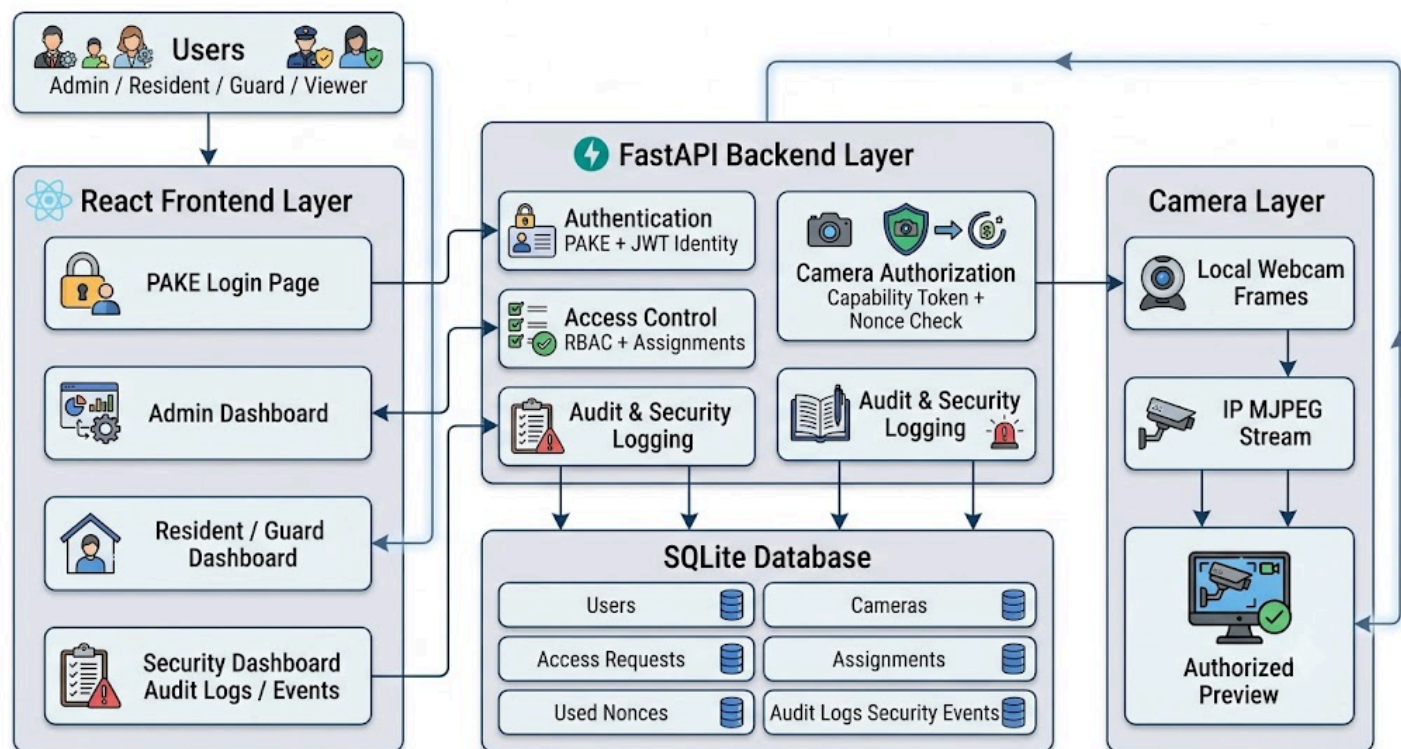


Figure 2.1: System Architecture of SecureCam

The architecture shows that the frontend handles user interaction, the backend enforces security decisions, the database maintains persistent access records, and the camera layer provides authorized frame or stream access only after the zero-trust chain is completed.

2.2 Authentication and Authorization Design

SecureCam follows a two-stage security model where authentication and authorization are treated as separate processes. During login, the user is authenticated using PAKE/SPAKE2, which allows secure password-based authentication without directly transmitting the password to the backend. After successful verification, the backend issues a JWT token containing the user's identity and role. However, this JWT is used only for identity verification and does not directly grant camera viewing permission.

Authorization is handled separately through role-based access control, admin-approved assignments, capability tokens, and nonce validation. Admin users can manage cameras, approve access requests, and view audit logs, while residents, guards, and viewers can access only the cameras assigned to them for a limited duration. Before any camera feed is displayed, the backend verifies the user's role, checks whether an active assignment exists, issues a camera-specific capability token, and validates a fresh nonce to prevent replay attacks. This ensures that camera access remains secure, temporary, and traceable.

2.3 Role-Based Access Control Design

SecureCam uses Role-Based Access Control (RBAC) to ensure that each user can perform only the actions allowed for their role. The system defines four main roles: admin, resident, security guard, and viewer. The admin has the highest privileges and can manage cameras, approve or reject access requests, revoke assignments, and monitor audit logs and security events. Limited users such as residents, guards, and viewers cannot directly access all cameras; they must have a valid assignment or approved access request.

This design follows the principle of least privilege, where users receive only the minimum access required for their purpose. Even after successful login, the role of the user is checked before allowing any sensitive action. For camera viewing, RBAC is combined with assignment validation, capability token checking, and nonce verification. This prevents unauthorized users from accessing camera feeds and ensures that every access decision is controlled, temporary, and traceable.

User Role	Main Permissions
Admin	Manage users/cameras, approve or reject requests, revoke access, view logs and security dashboard
Resident	Request camera access and view only assigned cameras
Security Guard	Request/view assigned surveillance cameras for limited duration
Viewer	View only specifically assigned cameras with restricted permissions

Table 2.1: Role-Based Access Control in SecureCam

2.4 Database Design and Backend Data Models

SecureCam uses an SQLite database to store the records required for authentication, authorization, camera access, replay protection, and audit monitoring. The main backend data models include users, cameras, access requests, assignments, used nonces, audit logs, and security events. These models allow the backend to verify identity, check permissions, manage temporary camera access, detect replay attempts, and maintain traceability.

The system also uses structured data objects such as JWT payloads, capability token payloads, request schemas, and response schemas to safely exchange information between the frontend and backend. Access requests and assignments represent the authorization state of a user, while used nonces act as one-time values to prevent replay attacks. Thus, the database tables and backend data models together form the core data structure of the SecureCam system.

Table Name	Description/Purpose
Users	Stores user login credentials, roles, and authentication-related metadata.
Cameras	Stores camera source details and configuration.
Access Requests	Tracks pending and historical camera access requests.
Assignments	Manages time-bound camera access permissions for users.
Used Nonces	Maintains a record of nonce values for replay attack prevention.
Audit Logs	Records system activity for traceability.
Security Events	Logs security-critical events like unauthorized access attempts.

Table 2.2: Database Tables and Backend Data Models Used in SecureCam

2.5 Flow Chart of the Proposed System

The system flow begins when a user opens SecureCam and attempts to log in. The login request is verified through PAKE/SPAKE2 authentication, and only after successful verification is a JWT identity token issued. The user is then redirected based on role. Admin users can manage cameras, approve access requests, and view logs, while residents, guards, and viewers must have an active assignment before accessing any camera feed.

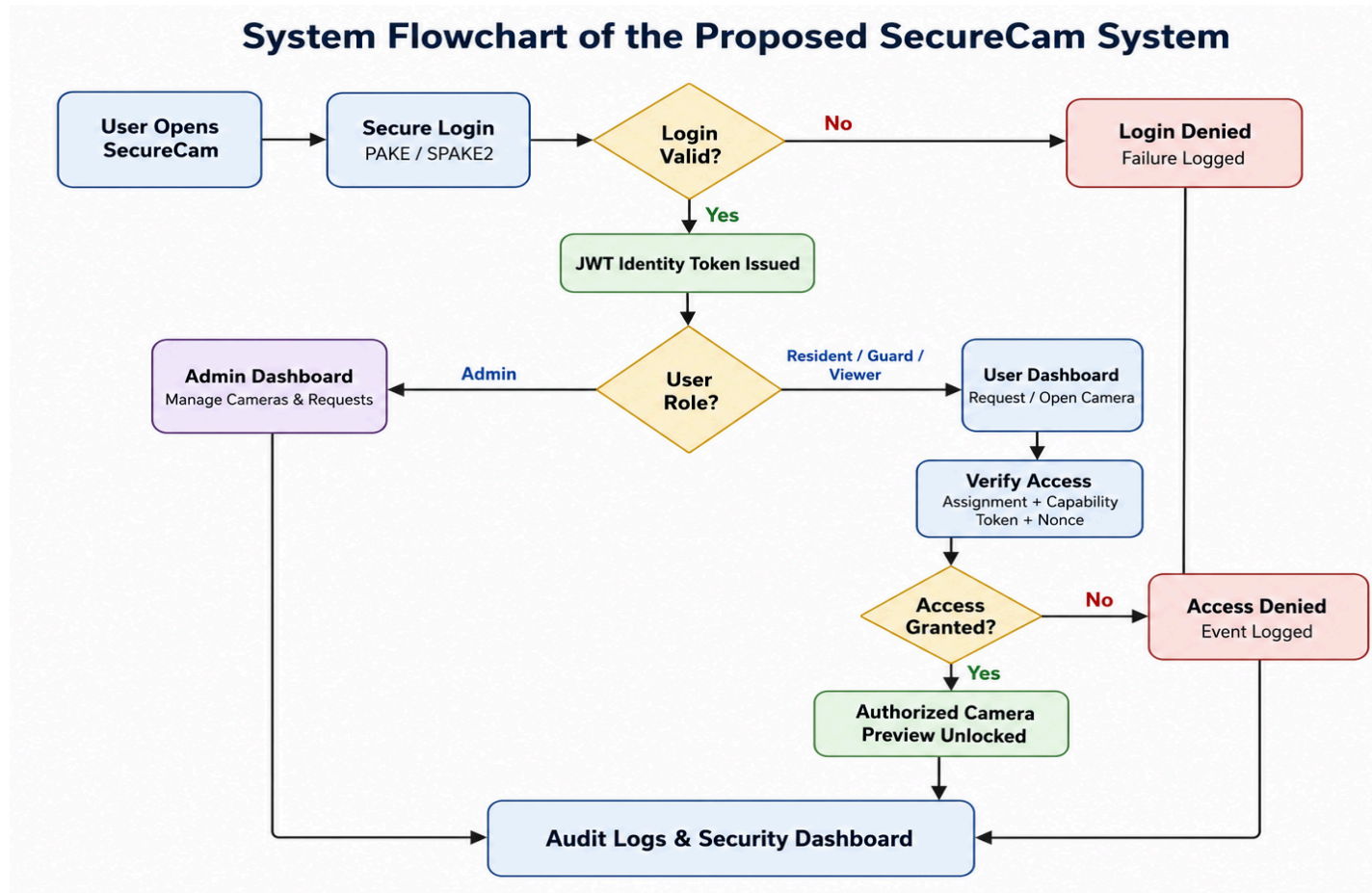


Figure 2.2: System Flowchart of the Proposed SecureCam System

2.6 Description of the Flow Chart

The flow starts when a user accesses the SecureCam web application and performs PAKE/SPAKE2-based login. If the authentication fails, the login attempt is rejected and recorded. If the authentication is successful, the backend issues a JWT identity token containing the user's identity and role.

After login, the system checks the user role. Admin users are allowed to manage cameras, approve or reject access requests, revoke assignments, and monitor audit logs. Limited users such as residents, guards, and viewers cannot directly view cameras; they must either request access or use an already approved active assignment.

When a limited user attempts to open a camera, the backend checks whether a valid assignment exists. If not, access is denied or kept pending. If the assignment is valid, the backend issues a camera-specific capability token and validates a fresh nonce. Reused nonces are rejected as replay attacks, while valid requests unlock the authorized camera preview and are recorded in the audit/security logs.

CHAPTER 3

IMPLEMENTATION DETAILS

3.1 Implementation Approach

SecureCam is implemented as a web-based full-stack application using a React and TypeScript frontend, a FastAPI backend, and an SQLite database. The implementation focuses on building a practical zero-trust surveillance access workflow where login, role verification, camera authorization, replay protection, and audit logging are handled as separate stages instead of treating login as direct camera permission.

The backend is developed using FastAPI because it provides fast API development, clear routing, request validation, and easy integration with authentication and database modules. SQLite is used for storing users, cameras, access requests, assignments, used nonces, audit logs, and security events. The PAKE authentication flow is implemented using SPAKE2 through a Node-based bridge, while JWT tokens are used only for identity and role verification after successful login.

The frontend is built using React and TypeScript to provide separate dashboards for admin, resident, security guard, and viewer roles. Admin users can manage cameras, approve or reject access requests, revoke assignments, and monitor logs. Limited users can request camera access and view only the cameras assigned to them for a specific duration. Camera preview is handled using local webcam frame upload and MJPEG-based streaming support for demonstration.

The implementation is divided into modules such as authentication, PAKE bridge, database models, request/response schemas, audit logging, camera access handling, and API routing. This modular approach makes the system easier to test, maintain, and extend. It also ensures that security-critical operations such as capability token validation, nonce replay detection, and access logging remain centralized in the backend.

3.2 Modular Design and Code Organization

The SecureCam codebase is organized around a clear separation of concerns, where each module handles a specific part of the authentication, authorization, camera access, or logging workflow. This makes the system easier to understand, test, maintain, and extend.

- **main.py:** Acts as the main FastAPI application file. It defines the core API routes for login, access requests, assignments, camera access, capability validation, audit logs, and security dashboard operations.
- **auth.py:** Handles authentication-related logic such as password verification, JWT token creation, current user identification, and role-based access checks.
- **pake_bridge.py:** Connects the FastAPI backend with the SPAKE2-based PAKE implementation. It allows the system to perform secure PAKE login without directly transmitting the password.
- **models.py:** Defines the database models such as users, cameras, access requests, assignments, used nonces, audit logs, and security events.
- **schemas.py:** Contains request and response schemas used by the API. These schemas ensure that data exchanged between the frontend and backend follows a proper structure.

- **audit.py:** Handles audit logging and security event generation for actions such as login success, login failure, access approval, camera view, replay attack detection, and unauthorized access attempts.
- **seed.py:** Provides initial sample data such as users, roles, and camera records required for testing and demonstration.

The frontend is organized into React components and pages for login, admin dashboard, resident or guard dashboard, camera preview, and security monitoring. This modular structure keeps the user interface separate from backend security logic. Overall, the design ensures that authentication, authorization, database handling, replay protection, camera preview, and audit monitoring remain independent but connected through well-defined API calls.

3.3 PAKE Authentication Implementation

The authentication module of SecureCam is implemented using Password-Authenticated Key Exchange (PAKE), specifically the SPAKE2 protocol. PAKE is used to improve the security of the login process by ensuring that the user's password is not directly transmitted to the backend during authentication. Instead of sending the plain password, the client and server perform a secure exchange that allows the backend to verify the login attempt without exposing the password over the network.

In the implementation, the FastAPI backend communicates with a Node-based PAKE bridge that handles the SPAKE2 authentication process. Once the PAKE verification is successful, the backend issues a JWT identity token to the user. This token contains basic identity information such as the user ID and role, and is used by the backend to recognize the user in later API requests.

The JWT token is not treated as direct camera access permission. It only proves that the user has successfully logged in. For viewing camera feeds, the system performs additional checks such as role validation, active assignment verification, capability token generation, and nonce validation. This separation ensures that authentication and camera authorization remain independent.

All login outcomes are recorded through the audit and security logging module. Successful logins are logged as normal authentication events, while failed login attempts are recorded for monitoring and traceability. This makes the authentication process secure, auditable, and suitable for a zero-trust surveillance access system.

3.4 Capability Token and Nonce Replay Protection

SecureCam implements capability tokens to ensure that camera access is granted only for a specific user, specific camera, and limited time period. After a user is authenticated and has an active admin-approved assignment, the backend issues a camera-scoped capability token. This token contains details such as user ID, camera ID, assignment ID, permission type, and expiry time.

The capability token is not issued immediately after login. It is generated only when the backend confirms that the user has the correct role and a valid active assignment for the requested camera. This prevents users from accessing cameras only by using a JWT token. The JWT proves identity, while the capability token proves temporary camera permission.

To prevent replay attacks, SecureCam uses nonce-based validation. A nonce is a one-time random value generated during camera authorization. When the frontend sends a capability validation request, the backend checks whether the nonce has already been used. If the nonce is fresh, access is allowed and the nonce is stored in the Used Nonces table.

If the same nonce is reused, the backend rejects the request and records it as a replay attack security event. This ensures that captured or repeated authorization requests cannot be used again to unlock camera feeds. Thus, capability tokens and nonce validation together provide secure, temporary, and replay-resistant camera access.

3.5 Camera Access and Streaming Implementation

The camera access module of SecureCam ensures that camera feeds are displayed only after proper authorization. A user cannot directly open a camera feed after login. When a resident, guard, or viewer attempts to access a camera, the backend verifies the JWT identity token, checks the user role, confirms whether an active assignment exists, and then validates the camera-scoped capability token with a fresh nonce.

For demonstration, SecureCam supports camera preview through local webcam frame upload and MJPEG-based streaming. In local webcam mode, frames are captured from the browser and sent to the backend at regular intervals. In MJPEG mode, the backend acts as a controlled proxy for the camera stream. Although the current implementation is demo-focused rather than production-grade WebRTC streaming, it successfully shows that camera feeds are unlocked only after identity verification, assignment checking, capability validation, nonce replay protection, and audit logging.

3.6 Audit Logs and Security Dashboard Implementation

SecureCam includes an audit logging module to record important user and system activities. Events such as login success, login failure, access request creation, request approval, request rejection, assignment revoke, camera view, unauthorized access, and replay attack detection are stored in the database. These logs help maintain traceability and allow the administrator to review who accessed which camera and when.

The security dashboard displays important security-related events in a centralized view. It helps the admin monitor replay attacks, unauthorized access attempts, expired access, and other sensitive actions. This improves accountability and makes the system more reliable for surveillance environments where access history and incident tracking are important.

3.7 Coding Best Practices and Maintainability

The project follows a modular coding approach where authentication, database models, request schemas, PAKE integration, audit logging, and camera access logic are separated into different files. This improves readability and makes it easier to test, debug, and update individual parts of the system without affecting the entire application.

The backend uses structured API routes, request/response schemas, role checks, and centralized validation to reduce errors and improve maintainability. The frontend is organized into reusable React components and role-specific pages. Overall, the code structure supports future improvements such as WebRTC streaming, real AI-based detection, cloud deployment, and enhanced security monitoring.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Execution Results and Validation

SecureCam was evaluated by testing the complete zero-trust surveillance access workflow from secure login to authorized camera viewing. The system was executed using a React frontend, FastAPI backend, SQLite database, and SPAKE2-based PAKE authentication. The validation covered PAKE login, JWT identity token generation, role-based dashboard access, admin approval of camera requests, assignment-based camera authorization, capability token validation, nonce replay protection, and audit/security event logging.

PAKE Authentication: The login process was tested through the SecureCam cryptographic handshake sequence. The system completed client initialization, server challenge, key exchange, client confirmation, and server verification without directly transmitting the password. After successful authentication, the backend issued a JWT identity token for further protected API access.

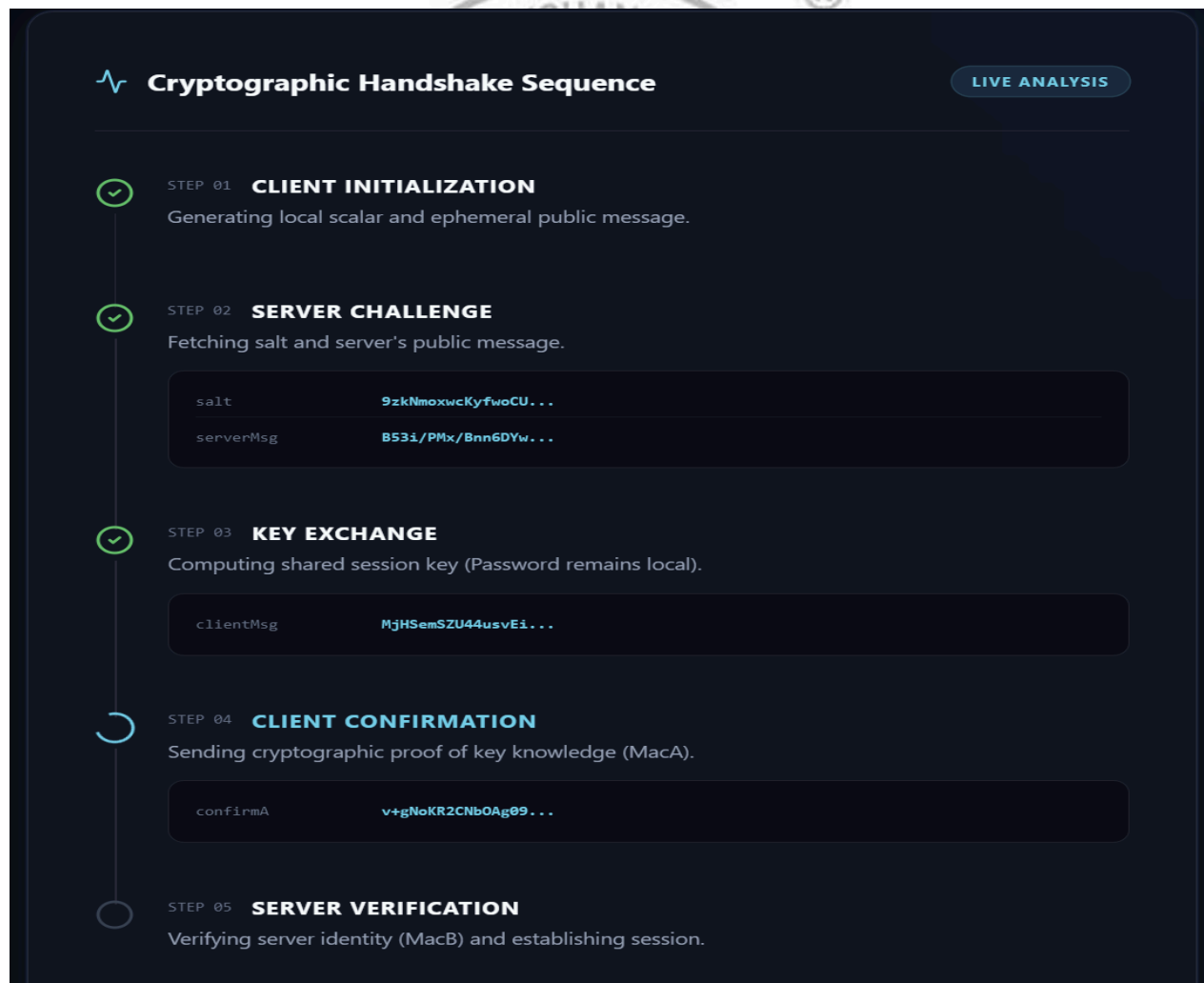


Figure 4.1: PAKE authentication handshake showing client initialization, server challenge, key exchange, and verification

Admin Access Management: The admin dashboard was used to configure cameras, view pending access requests, approve requests, and manage active assignments. The result confirms that camera permissions are not granted automatically, but are controlled by the administrator through a time-bound access workflow.

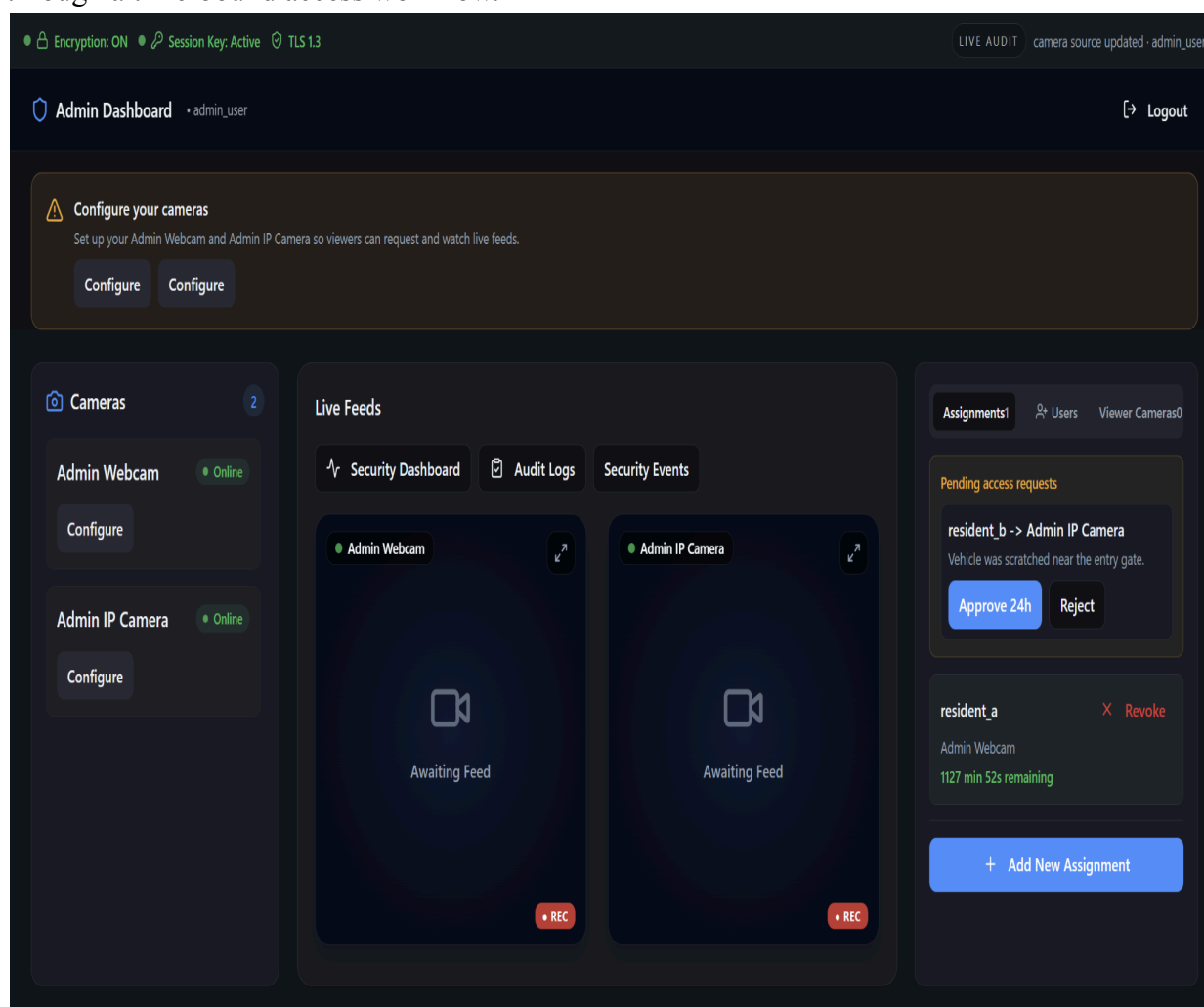


Figure 4.2: Admin dashboard showing camera management, pending access request, and active assignment

Authorized Camera Access: After the administrator approved the request and created an active assignment, the resident account was used to verify camera access from the limited-user side. The assigned camera appeared in the resident dashboard with a protected access state. Once the capability token was issued and validated with a fresh nonce, the camera preview was unlocked successfully.

The screenshot below shows that the resident user could access only the assigned camera, and the feed displayed the message “**Protected capability validated**”, confirming that camera access was granted only after assignment verification and capability validation.

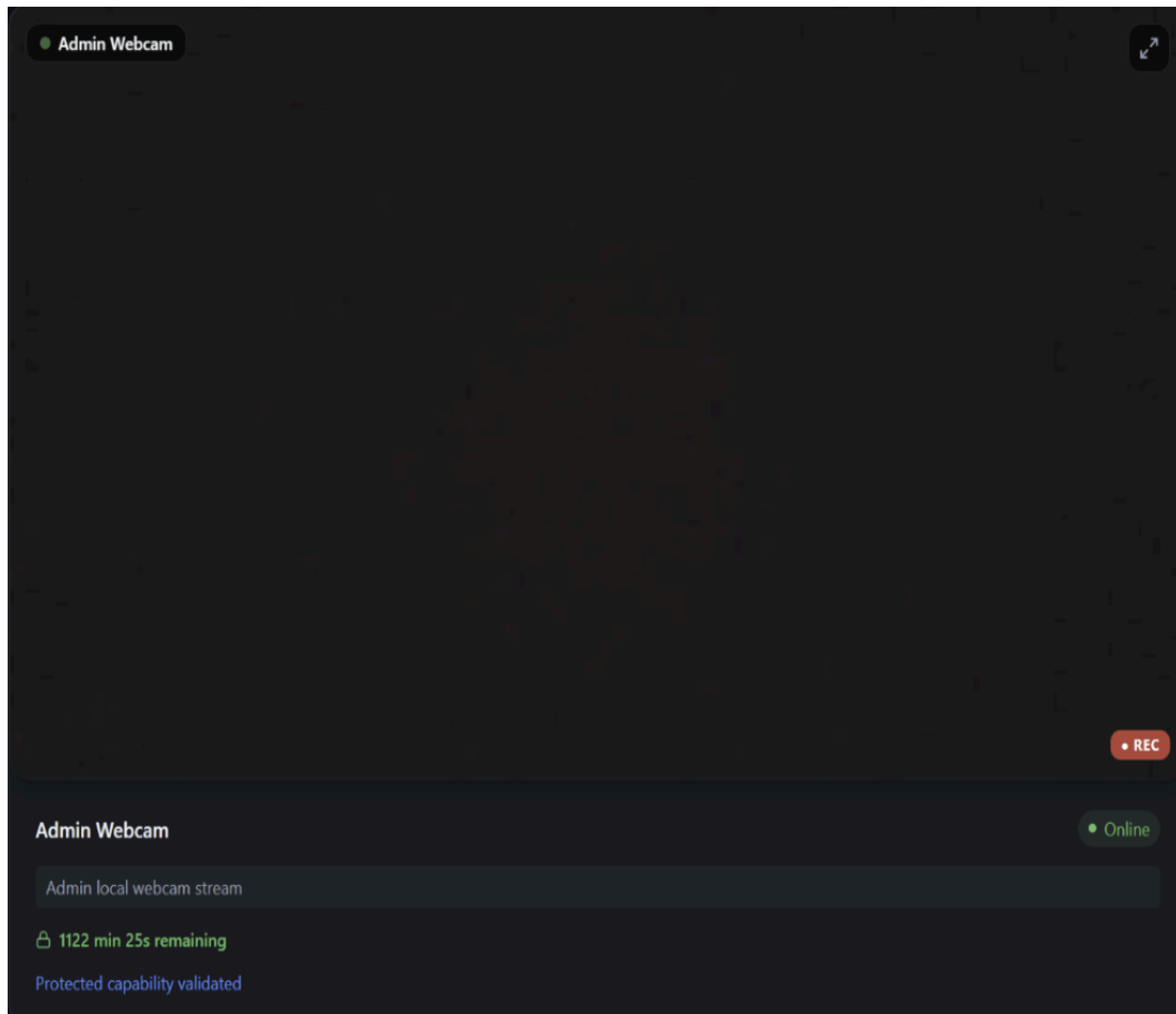


Figure 4.3: Authorized camera preview unlocked for resident after assignment and capability validation

Backend API Validation: The backend validation was performed to verify the complete zero-trust access chain through API-level testing. The test first confirmed that the backend health check was successful. Then, the admin account logged in and created a fresh assignment for `resident_a` on Camera 1. After this, the resident account logged in and requested a camera capability token.

The validation confirmed that the capability token was issued successfully and that a fresh nonce was accepted with HTTP status `200`. When the same nonce was reused, the backend rejected the request with HTTP status `409`, proving replay attack prevention. The latest security events also showed `REPLAY_ATTACK_DETECTED`, confirming that the replay attempt was logged correctly.


```
===== SECURECAM BACKEND VALIDATION =====

1) Backend Health Check
Backend status: ok

2) Admin Login
Admin login successful: admin_user

3) Create Fresh Assignment for resident_a on Camera 1
Assignment created for resident_a on Camera 1

4) Resident Login
Resident login successful: resident_a

5) Issue Camera Capability Token
Capability token issued for Camera ID: 1
Token type: capability
Permissions: VIEW
Expires at: 2026-06-07T20:28:41.637269

6) Detect Validate API Fields
Validate request fields: camera_id, capability_token, nonce

7) Validate Fresh Nonce
Fresh nonce validation HTTP status: 200

8) Replay Same Nonce
Replay validation HTTP status: 409
Replay rejected successfully

9) Latest Security Events
REPLAY_ATTACK_DETECTED | high | security | Replayed capability nonce rejected.
CAMERA_VIEW_STARTED | low | authorization | Camera View Started
LOGIN_SUCCESS | low | authentication | Login Success
ACCESS_GRANTED | low | authorization | Access Granted
LOGIN_SUCCESS | low | authentication | Login Success

===== VALIDATION COMPLETE =====
```

Figure 4.4: Backend API validation showing capability token issue, fresh nonce acceptance, replay attack rejection, and security event logging

The execution results confirm that SecureCam successfully separates authentication from camera authorization. Camera access is granted only after PAKE authentication, JWT identity verification, role checking, active assignment validation, capability token verification, and fresh nonce validation. This validates the main objective of building a secure, auditable, and replay-resistant zero-trust surveillance access system.

4.2 Security Test Cases and Edge Cases

Security test cases were performed to verify how SecureCam behaves under normal access, denied access, expired access, and replay attack conditions. Since surveillance feeds contain sensitive information, the system was tested not only for successful login and camera viewing, but also for situations where unauthorized or repeated access attempts should be blocked.

Test Case	Expected Result	Observed Result
Valid PAKE login	User is authenticated and JWT is issued	Login successful
Invalid login attempt	Login is denied and failure is logged	Authentication failure recorded
Admin approves access request	Time-bound assignment is created	Access granted to selected camera
User without assignment opens camera	Camera access is denied	Feed remains locked
User with valid assignment opens camera	Capability token is issued and feed is unlocked	Camera preview displayed
Expired or revoked assignment	Camera access is rejected	Access denied
Fresh nonce validation	Request is accepted	HTTP 200 returned
Reused nonce validation	Replay attack is detected	HTTP 409 returned and event logged
Admin checks security dashboard	Logs and security events are visible	Events displayed correctly

Table 4.1: Security Test Cases and Validation Results

The results show that SecureCam handles both normal and edge cases correctly. A successful login only provides identity verification and does not directly unlock camera feeds. Camera access is granted only after assignment verification, capability token validation, and fresh nonce checking. Reused nonces and unauthorized attempts are rejected and recorded in the audit/security logs, confirming that the system follows least-privilege and replay-resistant access control.

4.3 Replay Attack Detection and Access Revocation Results

Replay attack detection was tested by validating a camera capability token with a fresh nonce and then reusing the same nonce again. The first validation request was accepted with HTTP status 200, which confirmed that the nonce was fresh and valid. When the same nonce was reused, the backend rejected the request with HTTP status 409 and recorded the event as `REPLAY_ATTACK_DETECTED`. This proves that captured or repeated authorization requests cannot be reused to unlock camera feeds.

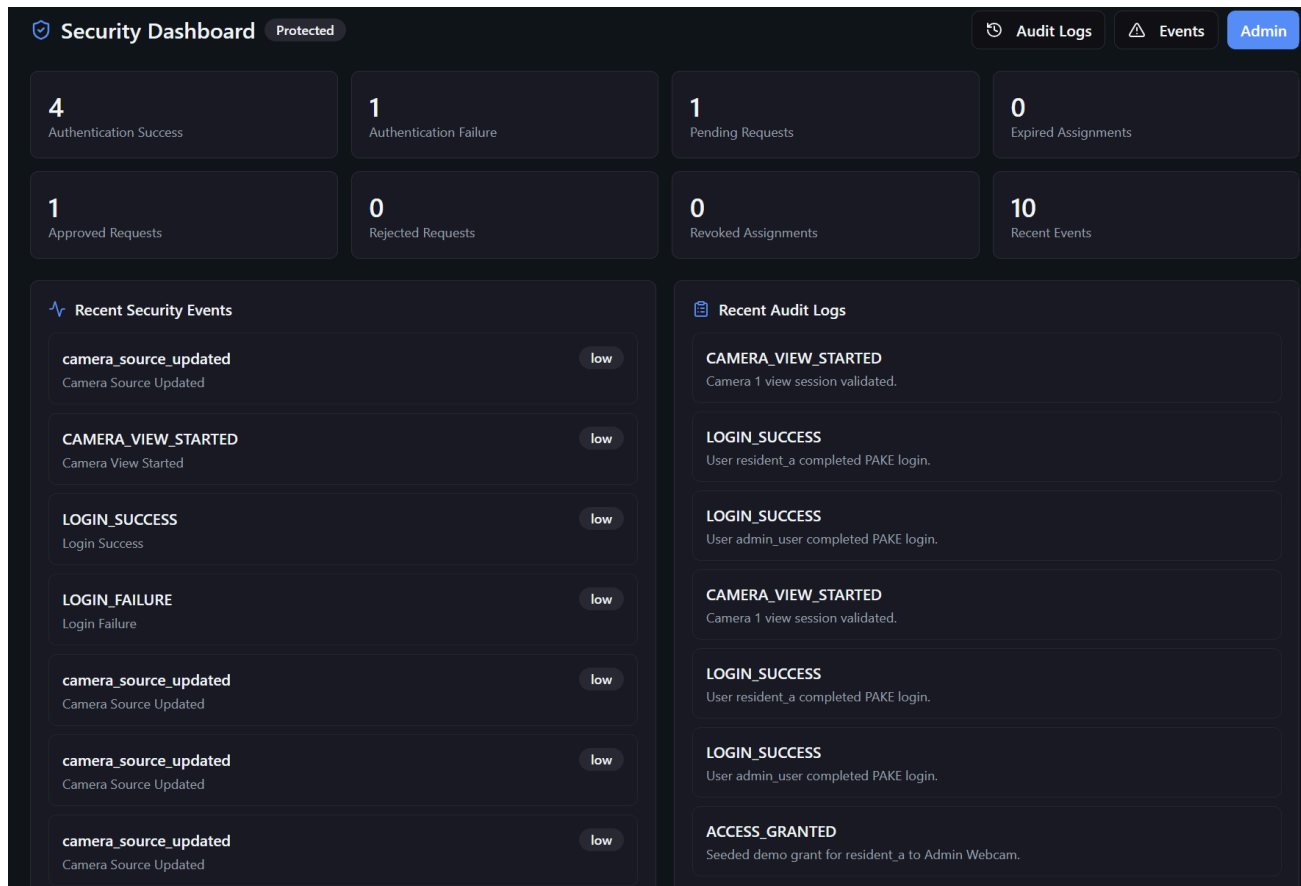


Figure 4.5: Security dashboard showing replay attack detection and logged security events

Access revocation was also tested through the admin dashboard by removing or revoking an active camera assignment. After revocation, the resident or limited user was no longer able to access the assigned camera because the backend did not find a valid active assignment. These results confirm that SecureCam provides temporary and controllable camera access, where permissions can expire automatically or be revoked manually by the administrator whenever required.

4.4 Comparative Analysis with Traditional Surveillance Systems

SecureCam was compared with traditional surveillance access systems to understand how the proposed zero-trust approach improves security. In many traditional systems, once a user logs in, the same session token may provide broad access to camera feeds. Such systems usually have limited camera-specific authorization, weak replay protection, and basic logging. SecureCam improves this by separating login from camera authorization and by requiring assignment validation, capability token verification, and fresh nonce checking before camera access is granted.

Feature	Traditional Surveillance System	SecureCam
Login Method	Username and password-based login	PAKE/SPAKE2-based secure authentication
Password Exposure Risk	Password may be sent directly during login	Password is not directly transmitted
Camera Access	Often granted after login	Granted only after assignment and capability validation
JWT Usage	May act as full access permission	Used only for identity and role verification
Access Duration	Often long-term or manually controlled	Time-bound assignments with expiry
Replay Attack Protection	Usually absent	Fresh nonce validation prevents replay
Access Revocation	Limited or delayed	Admin can revoke active assignments
Auditability	Basic activity logs	Detailed audit logs and security events
Security Model	Login-based access	Zero-trust, least-privilege access

Table 4.2: Comparison between Traditional Surveillance Systems and SecureCam

The comparison shows that SecureCam provides stronger protection than a normal login-based surveillance portal. It ensures that a valid login alone is not enough to view camera feeds. Instead, every camera access request is checked using user role, approved assignment, capability token, nonce freshness, and audit logging. This makes the system more secure, traceable, and suitable for environments where privacy and controlled surveillance access are important.

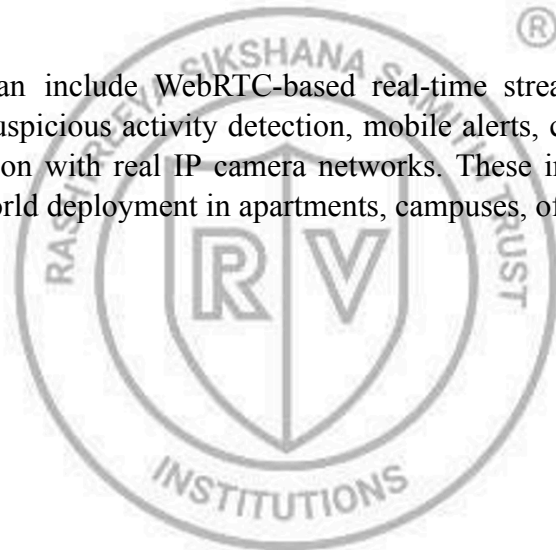
4.5 Discussion and Scope for Improvements

The results show that SecureCam successfully implements the main objective of a zero-trust surveillance access system. The system does not allow users to access camera feeds immediately after login. Instead, access is granted only after PAKE authentication, JWT identity verification, role-based access checking, active assignment validation, capability token verification, and fresh nonce validation. This confirms that authentication and camera authorization are properly separated.

The project also demonstrates important Network Programming and Security concepts such as secure authentication, least-privilege access, replay attack prevention, access revocation, audit logging, and security event monitoring. The backend validation results prove that a fresh nonce is accepted, while reuse of the same nonce is rejected and recorded as a replay attack. This makes the access workflow more secure and traceable compared to traditional login-based surveillance portals.

However, the current implementation is mainly designed as an academic prototype. The camera preview uses local webcam frame upload and MJPEG-based streaming, which is suitable for demonstration but not equivalent to production-grade video streaming. The system also does not currently include real AI-based object detection, mobile application support, cloud deployment, or large-scale multi-camera infrastructure.

Future improvements can include WebRTC-based real-time streaming, end-to-end encrypted video transfer, real AI-based suspicious activity detection, mobile alerts, cloud deployment, stronger database encryption, and integration with real IP camera networks. These improvements can make SecureCam more suitable for real-world deployment in apartments, campuses, offices, and restricted facilities.



CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 Summary of Findings

This project presented **SecureCam**, a zero-trust surveillance access system designed to secure camera access using PAKE authentication, role-based access control, capability tokens, nonce replay protection, and audit logging. The main finding is that user authentication alone should not be treated as permission to view sensitive surveillance feeds. Instead, camera access must be granted only after multiple verification stages. The implemented system successfully separated login from camera authorization. PAKE/SPAKE2 was used for secure authentication, JWT was used only for identity and role verification, and actual camera access required admin approval, active assignment validation, capability token issue, and fresh nonce checking. Backend validation confirmed that fresh nonce requests were accepted, while reused nonces were rejected and logged as replay attacks.

5.2 Real-World Applicability and Relevance

SecureCam is relevant to real-world environments such as residential apartments, college campuses, offices, parking areas, and restricted facilities where surveillance feeds must be protected from unauthorized access. In such places, users may need temporary access to specific cameras for valid reasons, but broad or permanent access can create privacy and security risks. The system provides a practical access control workflow where administrators can approve, revoke, and monitor camera permissions. Its use of time-bound assignments, replay-resistant authorization, and audit logs makes it suitable for scenarios where privacy, accountability, and controlled surveillance access are important.

5.3 Future Enhancements and Innovations

While the current implementation of SecureCam demonstrates PAKE-based authentication, zero-trust camera authorization, replay protection, and audit logging, the system can be improved further for real-world deployment. The following enhancements can make the system more scalable, secure, and intelligent:

- **WebRTC-based real-time streaming:** Replacing the current webcam frame upload and MJPEG preview with WebRTC would provide smoother, low-latency, production-grade camera streaming.
- **End-to-end encrypted video transfer:** Adding encryption for camera streams would ensure that video data remains protected between the camera source and the authorized viewer.
- **Real AI-based surveillance detection:** Integrating AI models for suspicious activity detection, object detection, or anomaly detection would make the system more proactive and intelligent.
- **Cloud and real IP camera integration:** Deploying the system on the cloud and connecting it with real CCTV/IP camera infrastructure would make SecureCam suitable for apartments, campuses, offices, and restricted facilities.

Together, these enhancements can evolve SecureCam from an academic prototype into a more complete and deployment-ready surveillance access control system.

REFERENCES

- [1] W. Ladd, “SPAKE2, a Password-Authenticated Key Exchange,” *RFC 9382*, Internet Research Task Force, Sept. 2023.
- [2] T. Taubert and C. A. Wood, “SPAKE2+, an Augmented Password-Authenticated Key Exchange Protocol,” *RFC 9383*, RFC Editor, Sept. 2023.
- [3] D. Temoshok *et al.*, “Digital Identity Guidelines: Authentication and Authenticator Management,” *NIST Special Publication 800-63B-4*, National Institute of Standards and Technology, July 2025.
- [4] O. Borchert, G. Howell, A. Kerman, S. Rose, M. Souppaya, *et al.*, “Implementing a Zero Trust Architecture: High-Level Document,” *NIST Special Publication 1800-35*, National Cybersecurity Center of Excellence, June 2025.
- [5] Cybersecurity and Infrastructure Security Agency, “Zero Trust Maturity Model Version 2.0,” *CISA*, Apr. 2023.
- [6] Canadian Centre for Cyber Security, “A Zero Trust Approach to Security Architecture,” *ITSM.10.008*, Government of Canada, Mar. 2023.
- [7] D. Fett, B. Campbell, J. Bradley, T. Lodderstedt, M. Jones, and D. Waite, “OAuth 2.0 Demonstrating Proof of Possession,” *RFC 9449*, Internet Engineering Task Force, Sept. 2023.
- [8] T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett, “Best Current Practice for OAuth 2.0 Security,” *RFC 9700*, Internet Engineering Task Force, Jan. 2025.
- [9] OWASP Foundation, “OWASP API Security Top 10 – 2023,” *Open Worldwide Application Security Project*, 2023.
- [10] K. Ragothaman, Y. Wang, B. P. Rimal, and M. Lawrence, “Access Control for IoT: A Survey of Existing Research, Dynamic Policies and Future Directions,” *Sensors*, vol. 23, no. 4, p. 1805, 2023.
- [11] C. Li, F. Li, C. Huang, L. Yin, T. Luo, and B. Wang, “A Traceable Capability-Based Access Control for IoT,” *Computers, Materials & Continua*, vol. 72, no. 3, pp. 4967–4982, 2022.
- [12] M. A. Bouras, B. Xia, A. O. Abuassba, H. Ning, and Q. Lu, “IoT-CCAC: A Blockchain-Based Consortium Capability Access Control Approach for IoT,” *PeerJ Computer Science*, vol. 7, p. e455, 2021.
- [13] R. Saha, G. Kumar, M. Conti, T. Devgun, T. H. Kim, M. Alazab, and R. Thomas, “DHACS: Smart Contract-Based Decentralized Hybrid Access Control for Industrial Internet-of-Things,” *IEEE Transactions on Industrial Informatics*, vol. 18, no. 5, pp. 3452–3461, 2022.
- [14] S. Pal and Z. Jadidi, “Protocol-Based and Hybrid Access Control for the IoT: Approaches and Research Opportunities,” *Sensors*, vol. 21, no. 20, p. 6832, 2021.
- [15] U. Tariq, I. Ahmed, A. K. Bashir, and K. Shaukat, “A Critical Cybersecurity Analysis and Future Research Directions for the Internet of Things: A Comprehensive Review,” *Sensors*, vol. 23, no. 8, p. 4117, 2023.

